

Technical Requirements: Content Design Tooling

Detailing requirements to support Central Content Design's goals to improve how we measure and improve content quality.

Background

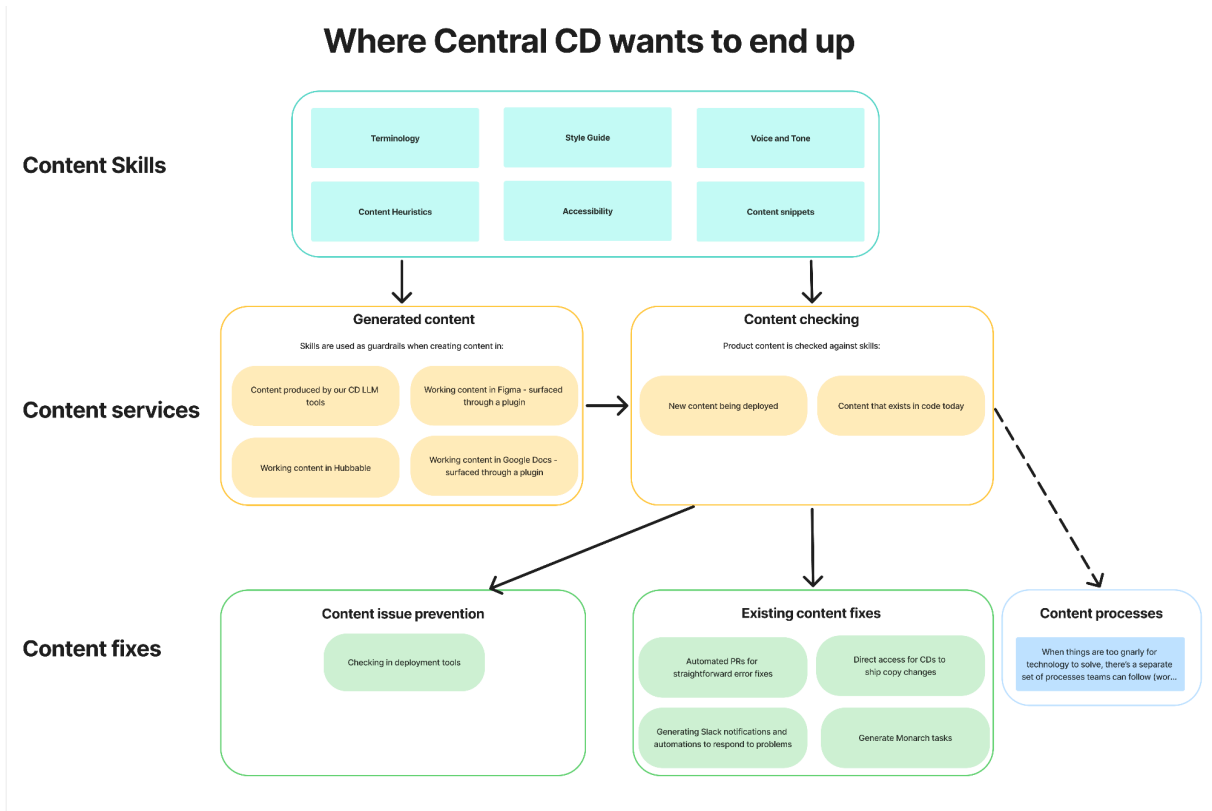
There are a number of  Centralized content quality infra projects running right now.

This list of requirements details the functionality needed to support these projects. Wherever possible, we have not specified which tools and technologies to use.

This document describes the infrastructure we need. It's structured in three layers:

1. **Content skills:** shared knowledge bases (terminology, style rules, accessibility standards, etc.) that any service can call on
2. **Services that use those skills:** tools that check existing content, generate new content, or do both
3. **Fix and reporting mechanisms:** the channels through which problems get fixed and progress gets tracked

Note: this list of requirements is not exhaustive, but it's also not all essential! There will be things we can manage without and work around for a while. In the short term, the biggest gap and the biggest opportunities for impact are in the "Fix" category. The [MVP section](#) gives more detail on this



[Check out the FigJam version of this diagram](#), which contains a simplified list of the requirements below.

Universal requirements

These requirements apply to any content tools we might develop:

- Tools can be accessed and actions triggered by Content Designers, using the standard tools and permissions available to them
- Provide a web UI to access and call information from content services - not all users are command line users.
- Where we use non-deterministic tools (like LLMs), we need predictable structures for outputs, with clear fallbacks, and strong guardrails.
- We should not use non-deterministic tools for tasks that need to be measured or completed consistently (like using specific approved terms, or measuring content quality).

- Any generative content tools should have HubSpot context; **content** context, like voice/tone/terms, and **product** context like "what Hub is this part of", "what related settings might you need to change", etc.

1. Content skills

Skills are shared knowledge bases that define what good content looks like. They need to be stored in structured, machine-readable formats so that any service can call on them. Each skill should also be accessible as a callable service, so it can be consumed by the terminology checker, Hubbable, Sidekick, Figma plugins, or any future tool.

Content skills need to be easy for content designers to update and maintain as we change standards and guidelines over time.

Writer is currently our system of record for terminology, style rules, and real-time content checking. Here are the features we would need to replicate if we stopped using Writer at HubSpot:

Terminology

- Store, create, update, and delete terms in a central database that our tools can read from; this [example export](#) from Writer contains all the fields in Writer's database.
- Support term metadata; current fields include:

None

Reference ID

Term

Type (e.g. Don't Use, Use carefully, Approved)

Part of speech (e.g. noun, adjective, verb, etc)

Case sensitive (True/False)

Capitalize at sentence start (True/False)

Autocorrect wrong capitalizations (True/False)

Description

Do example

Don't example
Common mistake
Tags
Replace with
Created by
Edited by
Created date
Edited date

- We would also like to add metadata in future - we've considered things like usage notes, synonyms, variants, translation, considerations, product line, owner.
- A simple process for updating and editing stored terms, definitions and metadata.
- Non-content-designers can suggest new terms to document, which content designers will run by a panel of approvers before updating the database.
- Apply bulk updates (for example, applying new tags to multiple terms at once, deleting a group of terms that have been deprecated, or bulk-importing a long list of new definitions that have been standardised).

Style guide

- Store and maintain HubSpot's [writing style guide](#) rules (voice and tone, punctuation, formatting, inclusiveness, word choice; also [corporate-level rules](#)) in a format our tools can check against.
- Style guide rules are stored in a way that's structured and accessible, so that they are easily understood by humans as well as machines (without the need to maintain separate human-facing and machine-facing versions).
- Beyond pure style checking, identify where simpler or clearer wording would be appropriate, based on things like sentence length, structure, long noun phrases, etc.
 - Writer provides Grammarly-style feedback to highlight complex or unnecessary content - we aim for Grade 6 reading level - and suggest better alternatives.

Voice and tone

- Store [voice and tone](#) rules as a distinct skill, separate from style guide rules. These cover how HubSpot sounds (friendly, clear, human) rather than specific formatting or word-choice rules.

Content heuristics

- Store the criteria we use to judge whether content is good (our [CD heuristics](#)). These are higher-level than terminology or style — things like "does this error message help the user recover?" or "is this label specific enough?"
- These checks will often need AI rather than pattern matching, because they require judgement. Any AI-powered checks need access to enough HubSpot context (our rules, our product terminology, what part of the product the content lives in) to make good assessments.

Accessibility

- Not for the initial build, but the architecture should support adding accessibility as a skill later, so that we can scan product content and check whether it meets whichever accessibility standards we prioritise and document at HubSpot.
- [Analysis shows](#) that WCAG 2.1 checks are feasible from string content alone: vague link text ("click here"), visual-only instructions ("see the form on the right"), colour as sole indicator ("click the red button"), generic error messages ("something went wrong"), ALL CAPS strings, and mouse-only instructions ("hover over").
- There could be useful overlap with our localization and i18n processes and systems here, too.

Content snippets

- Writer manages reusable text fragments — approved bits of copy that should be used consistently (e.g. standard error messages, legal language, feature descriptions). People and LLMs can insert these while writing to build consistency and ensure clarity.

- We'd need a way to store, edit, search, tag, and insert approved snippets, ideally accessible from writing and designing tools, like Google Docs and Figma.

2. Content Services

Services are the tools that call on skills to do useful work. They fall into two groups: services that **check and assess** existing content, and services that **generate** new content. Both groups use the same underlying skills.

Checking and assessing content

These services take content as input, check it against one or more skills, and produce structured results.

Terminology Checker

Building on the [Terminology Checker](#) tool (and

 Terminology: 2026 Content Design Vision) we've been developing, these are the requirements to replicate that functionality. More information on the technical setup of the app is [in this thread](#), and [Kieran Forde](#) can give more context!

- Retrieve all user-facing content from all relevant repositories across HubSpot
 - Currently, we use a python script that uses the Linguine API to fetch strings and save them locally - the latest scan contained 4,205 items and was 118MB.
 - [Alia Curtis](#) is working on a real-time Elasticsearch index of Linguine strings; we could use this as the input source instead of manual string uploads.
- Check content against any of our services to identify where content does not meet our standards.
- Where we find content problems, store the string ID, text, repo, file path, product area, issue type, container type (such as a component, or a heading), suggested fix, status, owning team, first detected date, and last scanned date.

- Support regular whole-product scans (monthly minimum, weekly ideal) and retain historical results to track trends.
- Store and surface metadata about each scan in a database that can handle hundreds of thousands of records per scan - the latest scan shows 270,702 issues.
- Analyse content against a set of patterns ([more information about how this works](#)) to identify potential new terms that need to be documented, so that we can contact relevant teams and ask them to contribute new definitions to our terminology database - this will enable us to spot new issues and make content more consistent in future.
 - Examples of heuristics are: a the same words appear multiple times (e.g. “agent”), multiple capitalized words appear together (e.g. Knowledge Vault), action and nouns together (e.g. “Add tool”), branded terms (e.g. “Breeze Studio”)
- Display results in a table that users can search for specific terms and phrases, and filter by Hub, product area, feature, repo, category, and status.
- Categorise issues into “Fix” (does not match guidelines, direct replacement available), “Review” (does not match guidelines, multiple options available), and “Document” (looks like a term, needs to be documented).
- Group multiple mentions of the same word into a single row (e.g. Click, Clicks, click, clicking, clicks), to reduce noise when scrolling the table.
- Edit and build on the content scanning logic - up to now we’ve focused on terminology, but in future we might want to build the ability to check whether content meets accessibility, readability, or localisation standards (and those standards may change over time).
- Directly fix issues where a single replacement term is available and teams do not need to be consulted.

Real-time checking in authoring tools

- Writer can check content directly in certain apps; there are plugins for Figma, Google Docs, many websites/web apps in Chrome, and some desktop apps.
- We need content checking in Figma (where product designers write UI copy) and Google Docs (where longer content is sometimes drafted). These are the two highest-value integration points.

Checking in deployment tools

- Check all new PRs for content issues against our terminology and style guidelines, preventing or adding friction to a release if not compliant (like Bethbot did).
- Reporting instances where content has been deployed that doesn't meet guidelines, so that relevant teams can follow up.

All checking services should:

- Return structured, consistent results: what was checked, what skill it was checked against, what failed, severity, and suggested fix.
- Results need to be consistently measurable. Counts of each type of violation and scores against heuristics/accessibility standards all need to be reliable and repeatable, so we can track change over time.

Generating content

These services create new content, using skills as guardrails to make sure it's on brand and meets our standards.

- Writer currently generates content from prompts, rewrites text, adjusts tone, and summarises; all following HubSpot's rules. Our stack should support content generation grounded in our content skills.
- Content generated by Hubbable, Sidekick, or CD LLM tools should all be able to call on the same skills to ensure quality and consistency.
- Generative tools benefit from having both content context (voice, tone, terms) and product context (which Hub, what the feature does, what related settings exist). Content context comes from the skills; product context may need additional integrations.

3. Fix and reporting mechanisms

When a checking service finds problems, these are the channels for getting them fixed and tracking progress. Where possible, we should align with existing HubSpot processes for shipping product updates.

Fix mechanisms

Different issue types need different fix paths:

Issue type	Potential fix approach
Clear-cut errors Banned terms, deprecated names, casing	Auto-create a PR via Linguine or Sidekick. Team reviews and merges. Alternatively, ship fixes direct to production from content tools. Or create a Monarch campaign.
Needs review Ambiguous or context-dependent fixes	Slack the owning team with the issue and a suggested fix. They decide, and we can track whether the fix has been made or rejected. Or create a Monarch campaign.
New terms to document Term Checker has identified a pattern that looks like it could be an undocumented term	Flag for Content Design. Optionally draft a definition using AI. Prompt relevant teams to contribute, with resources and an easy path to submit to the terms database. Option to reject a potential term with a reason, so it isn't flagged again for a set period.
Complex/strategic e.g. a term means different things in different Hubs	Direct engagement with product teams.

All fix mechanisms should:

- Track what happens to each fix; when a PR is merged, declined, or ignored, the app's issue status should update automatically.
- Maintain an audit trail for any automated fixes, with alerts for affected teams to see what has changed and why
- Be reusable by other content quality tools, not just the terminology app.
- Where we identify potential terms and ask teams to create new definitions;
 - Provide resources to help them meet relevant standards and guidelines for documentation (such as workshops and AI tools to generate definitions)
 - Easy journey to submit updates directly to our terms database/service
 - Option to reject a potential term and give a reason, meaning that future mentions of this term will not be suggested for review for a certain period (e.g. a year)

Results storage

- Store all scan results and issue data in a database that can handle hundreds of thousands of records per scan (the latest scan found 270,702 issues).
- Each issue must capture: string ID and text, repo, file path, product area, issue type, suggested fix, status, owning team, first detected date, and last scanned date.
- Retain historical results to track trends over time.

Reporting

We’re still working on a set of metrics to establish what good quality content looks like. Broadly, to monitor progress and improve quality, we will need:

- Dashboard showing issues by product area: count, type, status, and trend over time.
- Filter by product area, team, repo, issue type, and status.
- CSV export for sharing outside the app.
- A content quality compliance score per product area that feeds into Kelly's content quality metrics work.
- Short visual summaries for quarterly leadership updates.
- Track rate of new content issues being introduced to the product, not just existing issues found.
- A simple, shared view of terminology issues across HubSpot, and for each product area.
- To track issues raised by the app; issues open, resolved, or rejected
- To monitor the rate of new content issues being introduced to the product

Minimum Viable Product

Here’s how our requirements are met currently, and where the biggest gaps and blockers lie:

Process	Current solution	Proposed MVP	Priority
Content design documentation: standards, terms	Writer terms database	Terms database on HubSpot infra	● Low (in the short term) - Writer still works, and we can use CSV exports as a workaround

Terminology checking	Content fetching script	Linguine Elasticsearch DB (nearly complete)	● Medium - script works, but it's clunky
	Terminology Checker App	Rebuild terminology checker on HubSpot infra	● Medium - app works, but results need manual effort to turn findings into fixes
Content fixes	Manual fixes in Linguine, or by messaging teams	Automate routing issues to teams/repos/Monarch	● High - huge effort needed to turn issues into fixes at scale
Measuring and tracking	No repeatable processes yet	Monitoring changes that happen	● Medium - still working on defining our plan for this